Day 161 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 24, Day 1 reference

Watch: Machine Learning Fundamentals: Bias and Variance (StatQuest with Josh Starmer) | Lecture 08 - Bias-Variance Tradeoff (Caltech, Learning From Data)

`day161_bias_variance_learning_curves.py`, segment by segment, in the order `main()` uses things. The parts (load_adult, split_rows, tree_matrix, onehot_matrix, fit_logistic, predict_logistic, brier, bias_variance) are copied unchanged into `day161_exercise.py`; Day 109's trees and boosting and Day 116's softmax regression are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: d = one file's columns, in a dictionary · ref = the rows that set codes and scalings · X, y = features and 0/1 targets · w = logistic weights (w[0] the intercept) · P = B models' predictions, B × rows · fi, va = fitting and validation rows

Segment 0 — Setup

Imports: earlier weeks' parts from other folders

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
sys.path.insert(0, os.path.join(ROOT, "week-16-classical-ml-trees"))
sys.path.insert(0, os.path.join(ROOT, "week-17-classical-ml-kernels-ensembles"))
with contextlib.redirect_stdout(io.StringIO()):
    import day109_exercise as d109
    import day116_exercise as d116
```

- `ROOT = os.path.dirname(HERE)` — the folder above this one: the repository, where every week's folder lives.
- `sys.path.insert(0, os.path.join(ROOT, "week-16-classical-ml-trees"))` — lets Python find Day 109's exercise file in Week 16's folder.
- `with contextlib.redirect_stdout(io.StringIO()):` — the imported files print their hand checks; this sends that text into a string nobody reads.

Segment 1 — The parts

load_adult and split_rows

CODE

```
def load_adult():
    out = []
    os.makedirs(DATA, exist_ok=True)
    for name in ("adult.data", "adult.test"):
        if not os.path.exists(os.path.join(DATA, name)):
            urllib.request.urlretrieve(ADULT_URL + name, os.path.join(DATA, name))
        rows = [[x.strip() for x in ln.split(",")] for ln in open(os.path.join(DATA, name),
            encoding="utf-8")]
        rows = [r for r in rows if len(r) == 15 and "?" not in r]
        d = {c: np.array([r[k] for r in rows]) for k, c in enumerate(COLS)}
        d["y"] = np.array([r[14].rstrip(".") == ">50K" for r in rows], int)
        out.append(d)
    return out
def split_rows(n, frac=0.2, seed=0):
    perm = np.random.RandomState(seed).permutation(n); nv = int(round(frac * n))
    return np.sort(perm[nv:]), np.sort(perm[:nv])
```

- `[[x.strip() for x in ln.split(",")] for ln in open(...)]` — a nested list comprehension: every line of the file, cut at commas, spaces trimmed.
- `if len(r) == 15 and "?" not in r` — whole rows only: the test file's first line is a note, and '?' marks a missing value.
- `{c: np.array([r[k] for r in rows]) for k, c in enumerate(COLS)}` — a dictionary comprehension: one array per column name.
- `r[14].rstrip(".") == ">50K"` — the test file writes '>50K.' with a full stop; rstrip removes it.
- `np.sort(perm[nv:]), np.sort(perm[:nv])` — the shuffled order cut in two, each part sorted back into file order.

tree_matrix and onehot_matrix

CODE

```
def tree_matrix(d, ref):
    cols = [d[c].astype(float) for c in NUM]
    for c in CAT:
        cats = np.unique(ref[c]); rate = {v: ref["y"][ref[c] == v].mean() for v in cats}
        cols.append(np.array([rate.get(v, ref["y"].mean()) for v in d[c]]))
    return np.stack(cols, 1)
def onehot_matrix(d, ref):
    cols = []
    for c in NUM:
        x, r = d[c].astype(float), ref[c].astype(float)
        if c.startswith("capital"):
            x, r = np.log1p(x), np.log1p(r)
        cols.append(((x - r.mean()) / r.std())[:, None])
    for c in CAT:
        cols.append((d[c][:, None] == np.unique(ref[c])[None, :]).astype(float))
    return np.hstack(cols)
```

- `rate = {v: ref["y"][ref[c] == v].mean() for v in cats}` — each category's share above $50K, computed on the reference rows only.
- `rate.get(v, ref["y"].mean())` — dict.get with a default: a category never seen in the reference gets the overall share.
- `np.log1p(x)` — log(1 + x): safe for the many zeros in capital gains.

- `(d[c][:, None] == np.unique(ref[c])[None, :])` — broadcasting a column against a row: True where the row has that category.
- `np.hstack(cols)` — joins the blocks side by side.

fit_logistic: Newton's method

MATH

$$g = A^T(p - y)/n + \lambda w; \quad H = A^T\, diag(p(1-p))\, A/n + \lambda I; \quad w \leftarrow w - H^{-1}g$$

CODE

```
def fit_logistic(X, y, lam=LAM, iters=50):
    A = np.c_[np.ones(len(X)), X]; w = np.zeros(A.shape[1])
    R = lam * np.eye(A.shape[1]); R[0, 0] = 0.0
    for it in range(1, iters + 1):
        p = 1.0 / (1.0 + np.exp(-A @ w))
        g = A.T @ (p - y) / len(y) + R @ w
        H = (A * (p * (1 - p))[:, None]).T @ A / len(y) + R
        step = np.linalg.solve(H, g); w = w - step
        if np.abs(step).max() < 1e-9:
            break
    return w, it
def predict_logistic(w, X):
    return 1.0 / (1.0 + np.exp(-(w[0] + X @ w[1:])))
```

- `np.c_[np.ones(len(X)), X]` — a column of ones in front, so w[0] is the intercept.
- `R[0, 0] = 0.0` — the penalty skips the intercept.
- `(A * (p * (1 - p))[:, None]).T @ A` — A^T diag(p(1–p)) A without building the diagonal matrix: each row scaled first.
- `np.linalg.solve(H, g)` — solves H · step = g; faster and more accurate than inverting H.
- `if np.abs(step).max() < 1e-9: break` — stop when no weight moves any more.

brier and bias_variance

MATH

$$Brier = [mean(p_{avg} - y)^2 - s^2/B] + s^2(B-1)/B$$

CODE

```
def brier(y, p):
    return float(np.mean((p - y) ** 2))
def bias_variance(P, y):
    B = len(P); var = P.var(0, ddof=1)
    bias_noise = np.mean((P.mean(0) - y) ** 2) - var.mean() / B
    return float(np.mean((P - y) ** 2)), float(bias_noise), float(var.mean() * (B - 1) / B)
```

- `P.var(0, ddof=1)` — variance down each column (across models), dividing by B – 1.
- `np.mean((P.mean(0) - y) ** 2) - var.mean() / B` — the average model's error, minus the part of the variance it still carries.
- `var.mean() * (B - 1) / B` — the variance in the same units as the Brier score, so the two parts add up exactly.

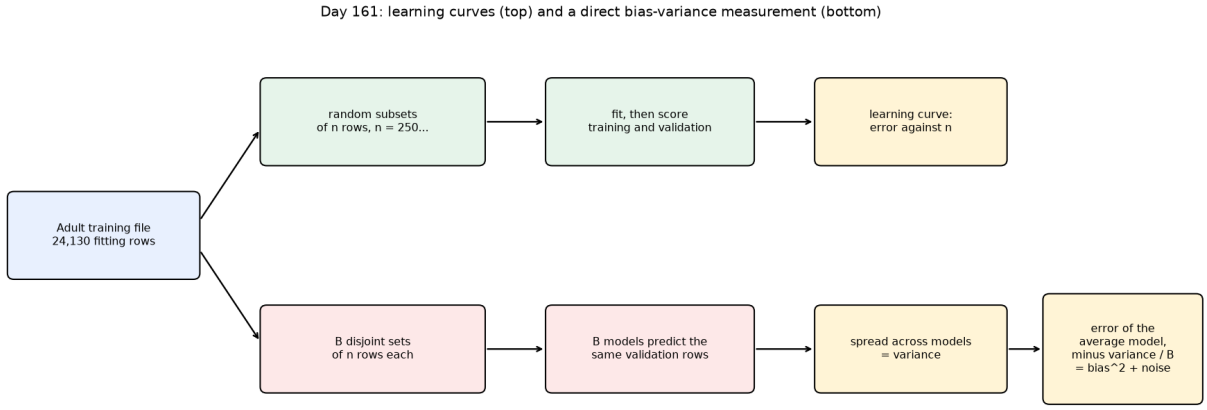
The three models as fit(X, y) -> predict

CODE

```
def logistic(X, y):
    w, _ = fit_logistic(X, y)
    return lambda Z: predict_logistic(w, Z)
def boosting(X, y):
    m, _ = d109.fit_boost(X, y, "log", GB["rounds"], GB["lr"], GB["depth"])
    return lambda Z: d109.sigmoid(d109.predict_boost(m, Z))
def deep_tree(X, y, depth=64):
    t = d109.grow_reg(X, y.astype(float), np.arange(len(y)), depth); d109.set_leaves(t, lambda i:
        y[i].mean())
    return lambda Z: d109.predict_reg(t, Z)
MODELS = {"logistic regression": logistic, "gradient boosting": boosting, "one deep tree":
    deep_tree}
```

- `return lambda Z: predict_logistic(w, Z)` — a function that remembers w: every model is used the same way, predict(X).
- `d109.set_leaves(t, lambda i: y[i].mean())` — each leaf's value is the share above $50K among its training rows.
- `MODELS = {"logistic regression": logistic, ...}` — functions stored in a dictionary, so one loop runs all three.

SYSTEM DIAGRAM



Learning curves (top) and the direct decomposition (bottom).

Segment 2 — The experiments

demo_curves: one loop over models, sizes and subsets

CODE

```
for name, fit in MODELS.items():
    X = feats(name, Xt, Xo); rows = []; t0 = time.time()
    for n in sizes:
        reps = REPEATS if n < len(fi) else 1
        r = []
        for s in range(reps):
            idx = np.sort(np.random.RandomState(1000 * s + n).choice(fi, n, replace=False))
            f = fit(X[idx], y[idx])
            pt, pv = f(X[idx]), f(X[va])
            r.append((brier(y[idx], pt), brier(y[va], pv), np.mean((pt > 0.5) != y[idx]),
                np.mean((pv > 0.5) != y[va])))
        r = np.array(r); rows.append((n,) + tuple(r.mean(0)) + (r[:, 1].std(),))
    out[name] = np.array(rows)
```

- `reps = REPEATS if n < len(fi) else 1` — the full set can only be drawn one way.
- `np.random.RandomState(1000 * s + n).choice(fi, n, replace=False)` — a subset of n fitting rows, with its own seed for every size and repeat.
- `(n,) + tuple(r.mean(0)) + (r[:, 1].std(),)` — tuples joined with +: the size, four averages, and the spread of validation scores.

disjoint_sets, demo_bv and demo_depth

CODE

```
def disjoint_sets(fi, n, B, seed):
    p = np.random.RandomState(seed).permutation(fi)
    return [np.sort(p[b * n:(b + 1) * n]) for b in range(B)]
    B = min(20, len(fi) // n)
    P = np.array([fit(X[idx], y[idx])(X[va]) for idx in disjoint_sets(fi, n, B, n)])
    rows.append((n, B) + bias_variance(P, y[va]))
```

- `p[b * n:(b + 1) * n]` — consecutive slices of one shuffled order never overlap.
- `len(fi) // n` — integer division: how many whole sets of n rows fit.
- `np.array([fit(...)(X[va]) for idx in ...])` — one row of validation predictions per training set: the B × rows array.

power_fit and demo_extrapolate

MATH

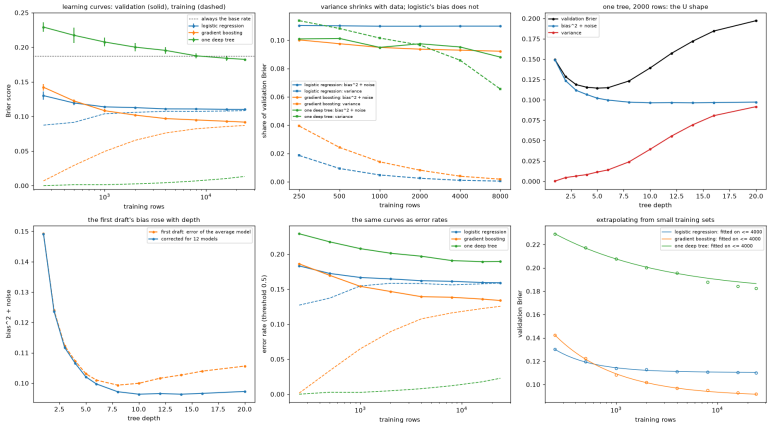
$error(n) = a + b\, n^{-c}$; *least squares for a, b at each c*

CODE

```
def power_fit(n, e):
    best = None
    for c in np.linspace(0.05, 1.5, 291):
        A = np.c_[np.ones(len(n)), n ** -c]; (a, b), *_ = np.linalg.lstsq(A, e, rcond=None)
        sse = float(np.sum((A @ np.array([a, b]) - e) ** 2))
        if best is None or sse < best[0]:
            best = (sse, a, b, c)
    return best[1:]
```

- `np.linspace(0.05, 1.5, 291)` — 291 exponents, 0.005 apart.
- `(a, b), *_ = np.linalg.lstsq(A, e, rcond=None)` — unpacking: the two coefficients, and *_ swallows the other returned values.
- `best[1:]` — the tuple without its first item (the error): (a, b, c).

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.linalg.solve(H, g)` — solve a linear system.
- `np.c_[a, B]` — join as columns.
- `x.var(0, ddof=1)` — sample variance down columns.
- `np.log1p(x)` — log(1 + x).
- `dict.get(key, default)` — look up with a fallback.
- `(a, b), *_ = f()` — unpack, discarding the rest.

End of Day 161 syntax reference. For the experiments and what they showed, see the Day 161 complete guide.